

Requirements + Actions

1. Must be implemented as a Windows service using the current version of in C#.

- `PowerPeriodService` is a `Worker` service project that can run as a console and windows service.

2. All trade positions must be aggregated per hour (local/wall clock time). Note that for a given day, the actual local start time of the day is 23:00 (11 pm) on the previous day.

Local time is London Local time for the day.

- `TradePositionData` project is responsible to request `PowerService`.
- Response from `PowerService` is then aggregated as required into `AggregatedPositionResult` object.
- This result object has dictionary with key as required wall clock time "23:00","00:00",...
- Only `DateTime` with `DateTimeKind` local is allowed.

3. CSV output format must be two columns, Local Time (format 24 hour HH:MM e.g. 13:00) and Volume and the first row must be a header row.

- `TradePositionPersistence` writes the `Csv` file with required header.

4. CSV filename must be `PowerPosition_YYYYMMDD_HHMM.csv` where `YYYYMMDD` is year/month/day e.g. 20141220 for 20 Dec 2014 and `HHMM` is 24hr time hour and minutes e.g. 1837. The date and time are the local time of extract.

- `TradePositionPersistence` creates required file name.
- `DateTime` at which Extract is requested is used in file name.

5. The location of the CSV file should be stored and read from the application configuration file.

- `appsettings.json` has field `CsvPowerPositionPath`

6. An extract must run at a scheduled time interval; every X minutes where the actual interval X is stored in the application configuration file. This extract does not have to run exactly on the minute and can be within +/- 1 minute of the configured interval.

- The interval time in minutes is read from `appsettings.json` - `IntervalInMinutes`
- `Task.Delay` in `Worker` takes this interval to trigger next extract.
- The next run does not worry if the earlier request is completed or not.

7. It is not acceptable to miss a scheduled extract.

- Retry is inbuilt in class libraries to get and to write extract.
- If post exponential retry there is no data, it is logged with Critical level.

8. An extract must run when the service first starts and then run at the interval specified as above.

- When console or service starts at T
 - It requests for Extract + Write
 - It schedules next Extract + Write at T + Interval

9. It is acceptable for the service to only read the configuration when first starting and it does not have to dynamically update if the configuration file changes. It is sufficient to require a service restart when updating the configuration.

- Configuration is read at start.

10. The service must provide adequate logging for production support to diagnose any issues.

- Each Extract + Write cycle is logged with a guid for proper traverse.
- Additionally, we log time taken for extract + write cycle.

How to run the app

1. As a console
 - a. Load solution
 - b. Get the required nuget packages
 - i. Moq
 - ii. Polly
 - iii. IO.Abstractions – this is for mocking IO operations.
 - c. Update appsettings.json for interval time and path
 - d. Update log4net.config for log path.
 - i. Default path is in the execution folder \logs
 - e. Hit F5
2. As a service
 - a. Load solution
 - b. Update the log file path, interval and csv paths
 - i. See that the csv path and log path has permissions for the service to write.
 - c. Publish and deploy the service
 - d. Start the service.